



# PREDICTIVE DIAGNOSTICS ENGINE

From Infrastructure Observer to Infrastructure Prophet

Zero-LLM

~0.05ms P95

Gemini A+

# FROM "WHAT IS" TO "WHAT WILL BE"

## PHASES 1–17: REACTIVE INTELLIGENCE

- ✓ Retrieve relevant infrastructure docs
- ✓ Decompose and answer complex queries
- ✓ Synthesize across conflicting sources
- ✓ Maintain conversational context

## PHASE 18: FORESIGHT

- ⚡ When will disk fill up?
- ⚡ Is memory pressure escalating or stable?
- ⚡ Is monitoring causing the spikes it observes?

REACTIVE



PREDICTIVE

# ZERO-LLM PREDICTIVE ENGINE

## THE PIPELINE

Prometheus (15s scrape)

→ metric\_snapshots table

→ PredictiveDiagnosticsEngine

→ |— OLS regression (closed-form)

→ |—  $R^2$  confidence gating

→ |— Step detection

→ |— TTA estimation

→ |— Observer effect

→ Glass Box Tab 7 (live feed)

## KEY DESIGN FACTS

### Zero ML libraries

Pure Python stdlib only

### Zero LLM calls

Deterministic arithmetic

### 100× under budget

0.05ms vs 5ms target

### Exact results

Same input = same output

# LIVE PREDICTIVE DIAGNOSTICS FEED

GLASS BOX ▶ Tab 7: Predictive Diagnostics

LIVE FEED

| Metric           | Trend      | R <sup>2</sup> | Delta | TTA    | Conf.  | Obs.                                                                                |
|------------------|------------|----------------|-------|--------|--------|-------------------------------------------------------------------------------------|
| disk_avail_lab01 | DECREASING | 0.99           | ***   | 278d   | HIGH   |  |
| disk_avail_lab02 | DECREASING | 0.97           | ***   | —      | HIGH   |                                                                                     |
| disk_avail_lab03 | DECREASING | 1.00           | ***   | —      | HIGH   |                                                                                     |
| cpu_rate_lab01   | STEP_UP    | 0.84           | ***   | —      | HIGH   |                                                                                     |
| cpu_rate_lab02   | INCREASING | 0.23           | ***   | —      | LOW    |                                                                                     |
| cpu_rate_lab03   | NOISY      | 0.41           | ***   | —      | LOW    |                                                                                     |
| mem_avail_lab01  | NOISY      | 0.01           | ***   | INSUF. | INSUF. |  |
| mem_avail_lab02  | STABLE     | 0.12           | ***   | INSUF. | INSUF. |                                                                                     |
| mem_avail_lab03  | NOISY      | 0.29           | ***   | INSUF. | INSUF. |                                                                                     |

R<sup>2</sup> 0.97–1.00 = HIGH confidence

R<sup>2</sup> 0.01–0.29 = INSUFFICIENT\_DATA

 = Observer effect detected

9 Prometheus series. R<sup>2</sup> proves the zero-LLM claim at a glance.



# CLOSED-FORM REGRESSION AT ~0.05ms P95

## WHAT IT COMPUTES

|                      |                                                 |
|----------------------|-------------------------------------------------|
| <b>Slope</b>         | Trend rate (units/sec)                          |
| <b>R<sup>2</sup></b> | Model fit quality (0–1)                         |
| <b>Trend</b>         | INCREASING / DECREASING / STABLE / NOISY / STEP |
| <b>TTA</b>           | Time-to-threshold (LOW → suppressed)            |
| <b>Confidence</b>    | HIGH / LOW / INSUFFICIENT_DATA                  |

## WHY IT'S FAST

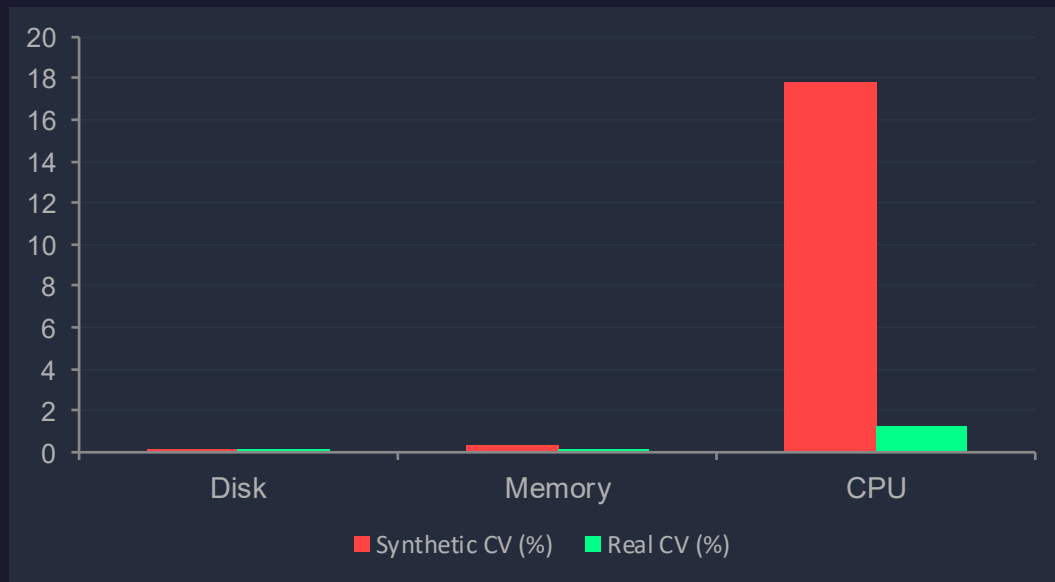
- ▶ Closed-form solution:  $O(n)$  arithmetic
- ▶ No model weights, no training, no inference
- ▶ Identical inputs → identical outputs
- ▶ No external dependencies

## PERFORMANCE



**100× HEADROOM**

# ⇄ REAL INFRASTRUCTURE: 10–65× MORE STABLE



## STABILITY FACTORS

Disk ~20× more stable

Memory ~6× more stable

CPU ~15× more stable

KEY INSIGHT: Synthetic baselines aren't simulations — they're conservative stress tests. If the engine handles 65× the noise of real production, it's production-ready.

# HONEST PREDICTIONS: ONLY WHEN EARNED

*A straight line through noisy data will always produce a slope. That doesn't mean the slope is meaningful.*

$$R^2 \geq 0.7$$

**HIGH confidence**

TTA computed

$$R^2 < 0.7$$

**LOW confidence**

TTA suppressed (—)

$$N < 20$$

**INSUFFICIENT\_DATA**

Prediction suppressed

## REAL DATA RESULTS

disk\_avail  $R^2=0.97-1.00$

HIGH

cpu\_rate  $R^2=0.23-0.84$

MIXED

mem\_avail  $R^2=0.01-0.29$

INSUFFICIENT

**Zero false urgency on stable production infrastructure.**

# THE OBSERVER EFFECT

THE A+ DIFFERENTIATOR

*The system recognizes when its own analytical overhead is responsible for the performance changes it is observing.*

## ALGORITHM

1. Load query timestamps (458 historical turns)
2. For each snapshot: was a query within  $\pm 30s$ ?
3. Partition: active periods vs baseline periods
4. Flag  if  $\text{active\_mean} - \text{baseline} > 1 \text{ std-dev}$

## LIVE EVIDENCE

### During queries:

CPU  $\rightarrow$  STEP\_UP (query-induced spike)

Disk  $\rightarrow$  DECREASING (real trend, unaffected)

### Observer flags:

disk\_avail\_proxmox-lab01

mem\_avail\_proxmox-lab01

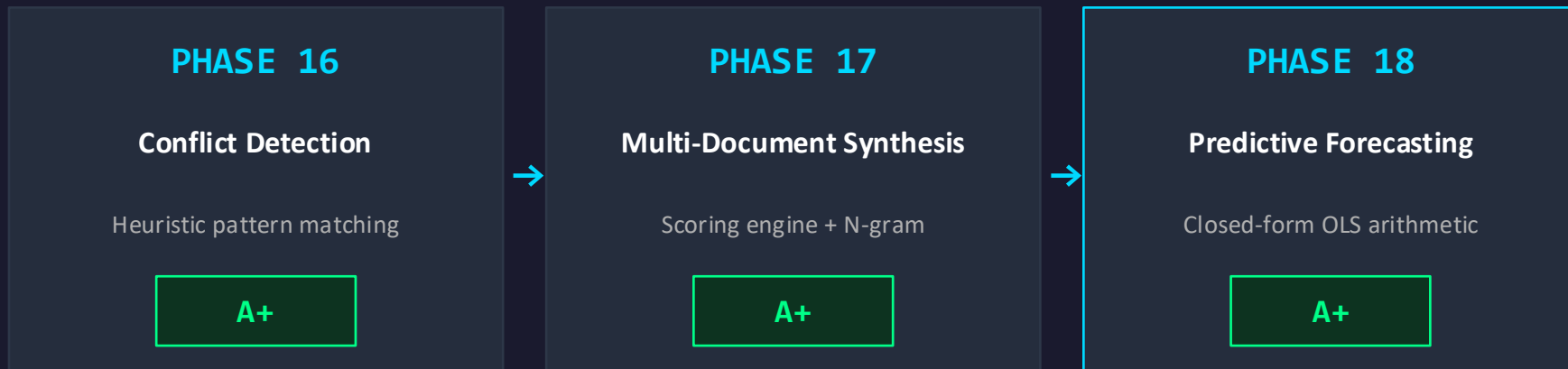
*"The system recognizes when its own analytical overhead is responsible for the performance changes it is observing."*

— Gemini External Validation

*Solves the Heisenbug of infrastructure monitoring*



# 3 PHASES ZERO LLM CALLS



*"The Zero-LLM constraint is a performance multiplier, not a limitation."* — **Gemini Validation**

**Semantic Intelligence** = LLMs for language and intent. **Analytical Intelligence** = Arithmetic for trends and data.



# FIVE PROBLEMS CAUGHT BEFORE PRODUCTION

| ISSUE                             | SESSION    | FIX                             |
|-----------------------------------|------------|---------------------------------|
| Empty database table              | Pre-launch | Audit before implementation     |
| X-axis overflow (Unix timestamps) | Week 2, S4 | Normalize to seconds-from-first |
| Spurious TTA on small windows     | Week 3, S7 | $N \geq 20$ sample guard        |
| Post-step slope contamination     | Week 3, S7 | Step honeymoon window reset     |
| Cache TTL flaky test              | Week 1, S1 | Widened assertion bound         |

## MEASURE TWICE, CUT ONCE

15 minutes of pre-flight database auditing prevented an entire Claude Code session from running on empty data.

# PHASE 18 BY THE NUMBERS

## PERFORMANCE

**~0.05ms**

P95 Latency

100× under 5ms budget

*"A staggering achievement." — Gemini*

## TEST COVERAGE

**808**

Total Tests

+90 this phase

*789/789 runnable @ 100%*

## STABILITY

**10–65×**

More Stable

Real infra vs synthetic baselines

*Zero false positives on production*

## ZERO-LLM STREAK

**3**

Consecutive Phases

100% deterministic

*Zero API cost in analytical layer*

# → FROM "THE WHAT" TO "THE WHY"

## PHASE 18

Prediction

*"Disk will be full in 12 minutes."*



## PHASE 19

Root Cause

*"Disk full in 12 min because Service X writes 50 errors/sec to logs."*

## THE BUILD

- ▶ Correlate metric trends (Phase 18) with cluster log events (Nomad, Consul)
- ▶ Apply multi-hop reasoning (Phase 15) to bridge logs to metrics
- ▶ Move from Vision → Root Cause Analysis

*"Use Phase 19 to see how far Deterministic Logic can go before Semantic Reasoning is truly required."*

— [Gemini External Validation](#)